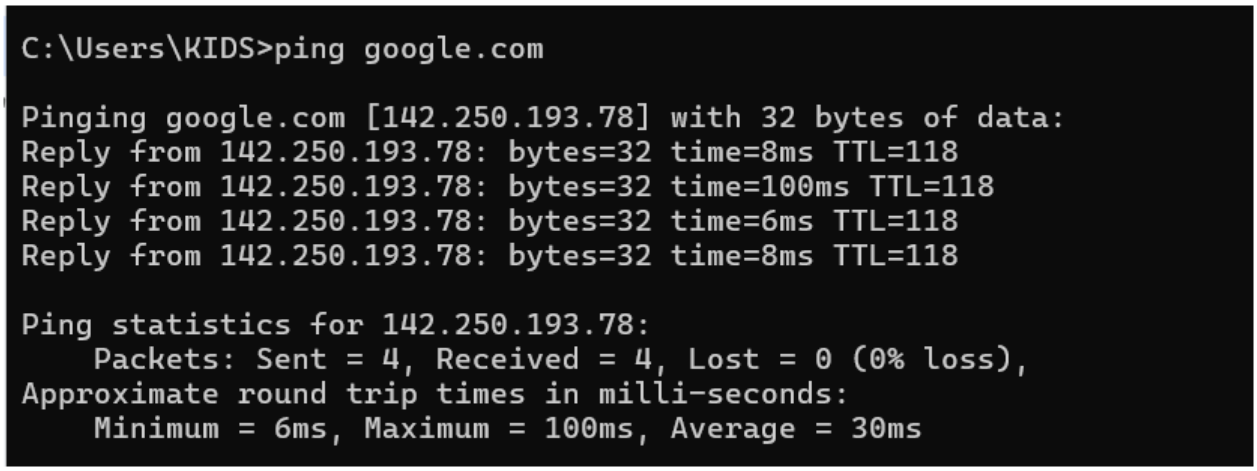
Keshav Mahavidyalaya
Univesity of Delhi

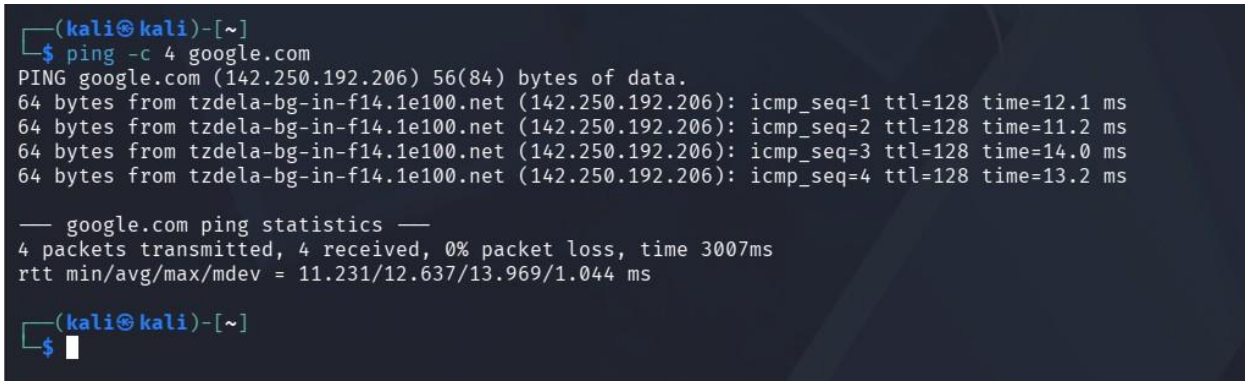
Information Security
Practical File

Name: Riya Garg
Course: B.Sc(Hons.) Computer Science
Exam Roll No.: 22035570072

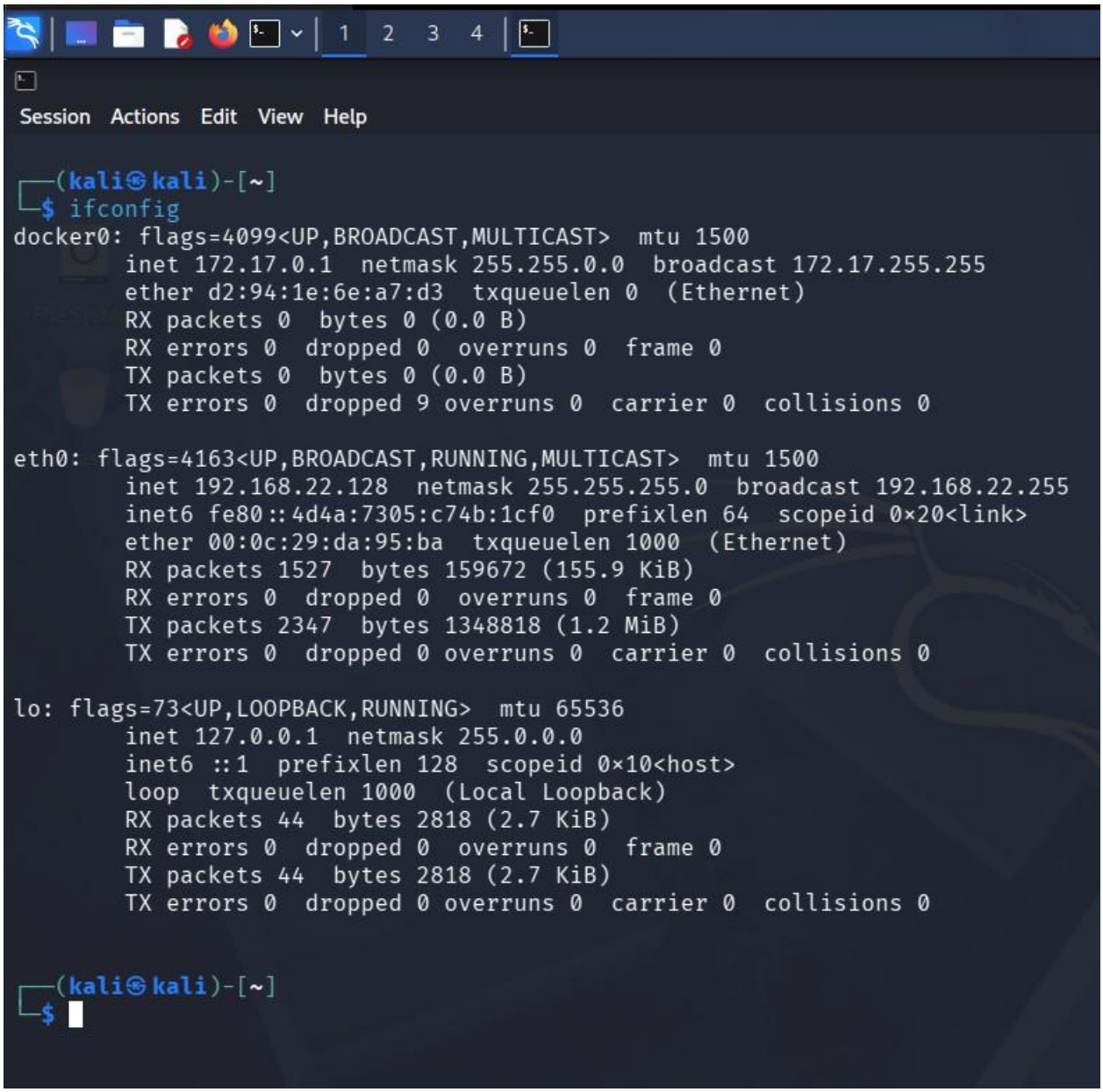
PRAC -1:

Screenshots:





```
C:\Users\KIDS>ipconfig

Windows IP Configuration


Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::f0a:d810:a912:ffe2%42
   IPv4 Address. . . . . . . . . . . : 192.168.116.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::36a0:98a7:60b6:36d7%23
   IPv4 Address. . . . . . . . . . . : 192.168.88.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : lan
   Link-local IPv6 Address . . . . . : fe80::3312:c9d8:c83f:7432%11
   IPv4 Address. . . . . . . . . . . : 172.16.3.249
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 172.16.3.1

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

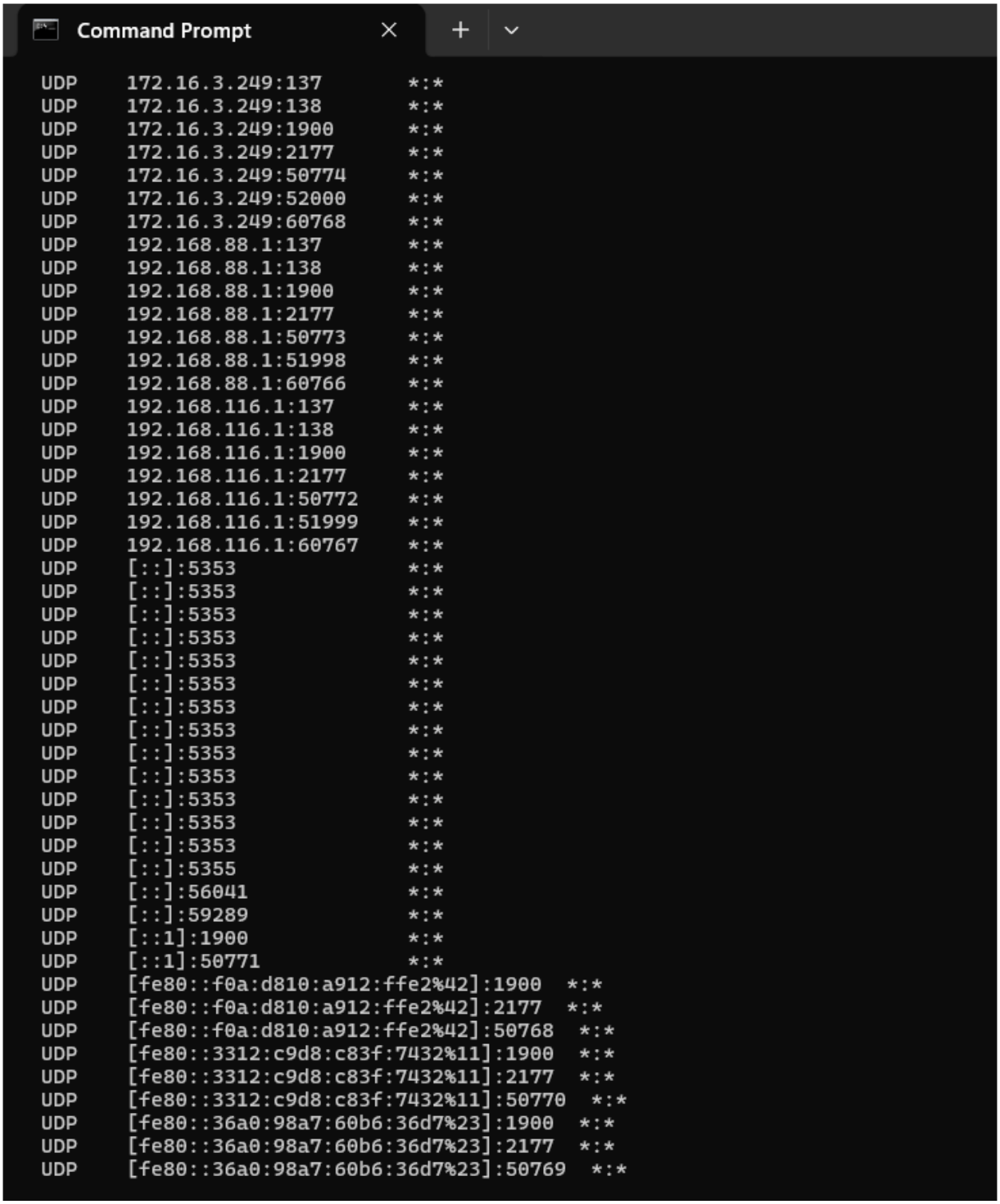

1 2 3 4
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
ether d2:94:1e:6e:a7:d3 txqueuelen 0 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 9 overruns 0 carrier 0 collisions 0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.22.128 netmask 255.255.255.0 broadcast 192.168.22.255
inet6 fe80::4d4a:7305:c74b:1cf0 prefixlen 64 scopeid 0×20<link>
ether 00:0c:29:da:95:ba txqueuelen 1000 (Ethernet)
RX packets 1527 bytes 159672 (155.9 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 2347 bytes 1348818 (1.2 MiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0×10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 44 bytes 2818 (2.7 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 44 bytes 2818 (2.7 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
(kali㉿kali)-[~]
$

```
C:\Users\KIDS>tracert

Usage: tracert [-d] [-h maximum_hops] [-j host-list] [-w timeout]
               [-R] [-S srcaddr] [-4] [-6] target_name

Options:
    -d                 Do not resolve addresses to hostnames.
    -h maximum_hops    Maximum number of hops to search for target.
    -j host-list       Loose source route along host-list (IPv4-only).
    -w timeout         Wait timeout milliseconds for each reply.
    -R                 Trace round-trip path (IPv6-only).
    -S srcaddr         Source address to use (IPv6-only).
    -4                 Force using IPv4.
    -6                 Force using IPv6.

C:\Users\KIDS>tracert google.com

Tracing route to google.com [142.250.193.78]
over a maximum of 30 hops:

  1     2 ms     3 ms     2 ms  i2e1.lan [172.16.3.1]
  2    13 ms     3 ms     9 ms  node103233116194.zeonet.co.in [103.233.116.194]
  3     *        *        *     Request timed out.
  4     *        *        *     Request timed out.
  5    20 ms   102 ms    10 ms  72.14.217.72
  6    14 ms    26 ms     7 ms  142.251.77.187
  7    19 ms    10 ms     4 ms  74.125.37.63
  8     7 ms     5 ms    10 ms  del11s16-in-f14.1e100.net [142.250.193.78]

Trace complete.
```

```
C:\Users\KIDS>arp -a

Interface: 172.16.3.249 --- 0xb
  Internet Address      Physical Address      Type
  172.16.3.1            32-de-4b-90-9b-75     dynamic
  224.0.0.2             01-00-5e-00-00-02     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 192.168.88.1 --- 0x17
  Internet Address      Physical Address      Type
  192.168.88.254        00-50-56-ee-6b-e7     dynamic
  192.168.88.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 192.168.116.1 --- 0x2a
  Internet Address      Physical Address      Type
  192.168.116.254       00-50-56-f0-7b-48     dynamic
  192.168.116.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

```
┌──(kali㉿kali)-[~]
└─$ arp -a
? (192.168.22.129) at <incomplete> on eth0
? (192.168.22.2) at 00:50:56:ed:e6:23 [ether] on eth0

┌──(kali㉿kali)-[~]
└─$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.22.129                   (incomplete)                              eth0
192.168.22.2             ether   00:50:56:ed:e6:23   C                     eth0
```

```
C:\Users\KIDS>netstat -an

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:902            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:912            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5040           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5432           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:12345          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49670          0.0.0.0:0              LISTENING
  TCP    127.0.0.1:11434        0.0.0.0:0              LISTENING
  TCP    127.0.0.1:52140        0.0.0.0:0              LISTENING
  TCP    172.16.3.249:139       0.0.0.0:0              LISTENING
  TCP    172.16.3.249:1551      142.250.122.94:443     ESTABLISHED
  TCP    172.16.3.249:5525      172.217.194.188:5228   ESTABLISHED
  TCP    172.16.3.249:11444     57.144.147.32:443      TIME_WAIT
  TCP    172.16.3.249:13968     23.210.213.75:443      ESTABLISHED
  TCP    172.16.3.249:14169     142.250.193.14:443     TIME_WAIT
  TCP    172.16.3.249:14972     142.250.193.78:443     ESTABLISHED
  TCP    172.16.3.249:18507     103.233.117.203:443    CLOSE_WAIT
  TCP    172.16.3.249:18508     23.217.111.137:443     CLOSE_WAIT
  TCP    172.16.3.249:18510     23.195.105.9:443       CLOSE_WAIT
  TCP    172.16.3.249:18514     23.11.39.161:80        ESTABLISHED
  TCP    172.16.3.249:18515     23.11.39.161:80        ESTABLISHED
  TCP    172.16.3.249:19475     104.18.39.21:443       ESTABLISHED
  TCP    172.16.3.249:20004     142.251.43.110:443     TIME_WAIT
  TCP    172.16.3.249:20531     192.178.134.94:443     TIME_WAIT
  TCP    172.16.3.249:25125     172.64.148.235:443     ESTABLISHED
  TCP    172.16.3.249:25739     142.250.122.94:443     ESTABLISHED
  TCP    172.16.3.249:27945     142.250.193.67:443     ESTABLISHED
  TCP    172.16.3.249:35925     142.250.193.35:443     TIME_WAIT
  TCP    172.16.3.249:38891     35.190.80.1:443        TIME_WAIT
  TCP    172.16.3.249:41364     57.144.147.32:443      ESTABLISHED
  TCP    172.16.3.249:43274     104.17.208.5:443       ESTABLISHED
  TCP    172.16.3.249:46836     104.18.36.252:443      ESTABLISHED
  TCP    172.16.3.249:48369     100.21.26.48:443       ESTABLISHED
  TCP    172.16.3.249:48676     54.192.142.120:443     ESTABLISHED
  TCP    172.16.3.249:48678     23.11.39.161:80        TIME_WAIT
  TCP    172.16.3.249:48679     131.253.33.203:80      TIME_WAIT
  TCP    172.16.3.249:49396     142.250.193.14:443     ESTABLISHED
  TCP    172.16.3.249:53094     52.98.88.66:443        ESTABLISHED
  TCP    172.16.3.249:53095     52.98.88.66:443        ESTABLISHED
  TCP    172.16.3.249:53446     172.188.155.25:443     ESTABLISHED
  TCP    172.16.3.249:53585     40.104.66.194:443      ESTABLISHED
  TCP    172.16.3.249:55937     142.250.193.35:443     TIME_WAIT
  TCP    172.16.3.249:56659     104.18.36.252:443      ESTABLISHED
  TCP    172.16.3.249:57433     104.18.24.232:443      ESTABLISHED
  TCP    172.16.3.249:58392     172.217.26.110:443     ESTABLISHED
```


Command Prompt
UDP 172.16.3.249:137 *:*
UDP 172.16.3.249:138 *:*
UDP 172.16.3.249:1900 *:*
UDP 172.16.3.249:2177 *:*
UDP 172.16.3.249:50774 *:*
UDP 172.16.3.249:52000 *:*
UDP 172.16.3.249:60768 *:*
UDP 192.168.88.1:137 *:*
UDP 192.168.88.1:138 *:*
UDP 192.168.88.1:1900 *:*
UDP 192.168.88.1:2177 *:*
UDP 192.168.88.1:50773 *:*
UDP 192.168.88.1:51998 *:*
UDP 192.168.88.1:60766 *:*
UDP 192.168.116.1:137 *:*
UDP 192.168.116.1:138 *:*
UDP 192.168.116.1:1900 *:*
UDP 192.168.116.1:2177 *:*
UDP 192.168.116.1:50772 *:*
UDP 192.168.116.1:51999 *:*
UDP 192.168.116.1:60767 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5353 *:*
UDP [::]:5355 *:*
UDP [::]:56041 *:*
UDP [::]:59289 *:*
UDP [::1]:1900 *:*
UDP [::1]:50771 *:*
UDP [fe80::f0a:d810:a912:ffe2%42]:1900 *:*
UDP [fe80::f0a:d810:a912:ffe2%42]:2177 *:*
UDP [fe80::f0a:d810:a912:ffe2%42]:50768 *:*
UDP [fe80::3312:c9d8:c83f:7432%11]:1900 *:*
UDP [fe80::3312:c9d8:c83f:7432%11]:2177 *:*
UDP [fe80::3312:c9d8:c83f:7432%11]:50770 *:*
UDP [fe80::36a0:98a7:60b6:36d7%23]:1900 *:*
UDP [fe80::36a0:98a7:60b6:36d7%23]:2177 *:*
UDP [fe80::36a0:98a7:60b6:36d7%23]:50769 *:*

```
┌──(kali㉿kali)-[~]
└─$ netstat -tunlp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:36309         0.0.0.0:*               LISTEN      -
```

```
┌──(kali㉿kali)-[~]
└─$ sudo netstat -tunlp
[sudo] password for kali:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:36309         0.0.0.0:*               LISTEN      1011/containerd
```

```
┌──(kali㉿kali)-[~]
└─$ whois google.com
   Domain Name: GOOGLE.COM
   Registry Domain ID: 2138514_DOMAIN_COM-VRSN
   Registrar WHOIS Server: whois.markmonitor.com
   Registrar URL: http://www.markmonitor.com
   Updated Date: 2019-09-09T15:39:04Z
   Creation Date: 1997-09-15T04:00:00Z
   Registry Expiry Date: 2028-09-14T04:00:00Z
   Registrar: MarkMonitor Inc.
   Registrar IANA ID: 292
   Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
   Registrar Abuse Contact Phone: +1.2086851750
   Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
   Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
   Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
   Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
   Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
   Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
   Name Server: NS1.GOOGLE.COM
   Name Server: NS2.GOOGLE.COM
   Name Server: NS3.GOOGLE.COM
   Name Server: NS4.GOOGLE.COM
   DNSSEC: unsigned
   URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-05-10T12:00:09Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar.  Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois
```

```
The data in MarkMonitor's WHOIS database is provided for information purposes,
and to assist persons in obtaining information about or related to a domain
name's registration record. While MarkMonitor believes the data to be accurate,
the data is provided "as is" with no guarantee or warranties regarding its
accuracy.

By submitting a WHOIS query, you agree that you will use this data only for
lawful purposes and that, under no circumstances will you use this data to:
  (1) allow, enable, or otherwise support the transmission by email, telephone,
or facsimile of mass, unsolicited, commercial advertising, or spam; or
  (2) enable high volume, automated, or electronic processes that send queries,
data, or email to MarkMonitor (or its systems) or the domain name contacts (or
its systems).

MarkMonitor reserves the right to modify these terms at any time.

By submitting this query, you agree to abide by this policy.

MarkMonitor Domain Management(TM)
Protecting companies and consumers in a digital world.

Visit MarkMonitor at https://www.markmonitor.com
Contact us at +1.8007459229
In Europe, at +44.02032062220
--
```

PRAC – 2 :

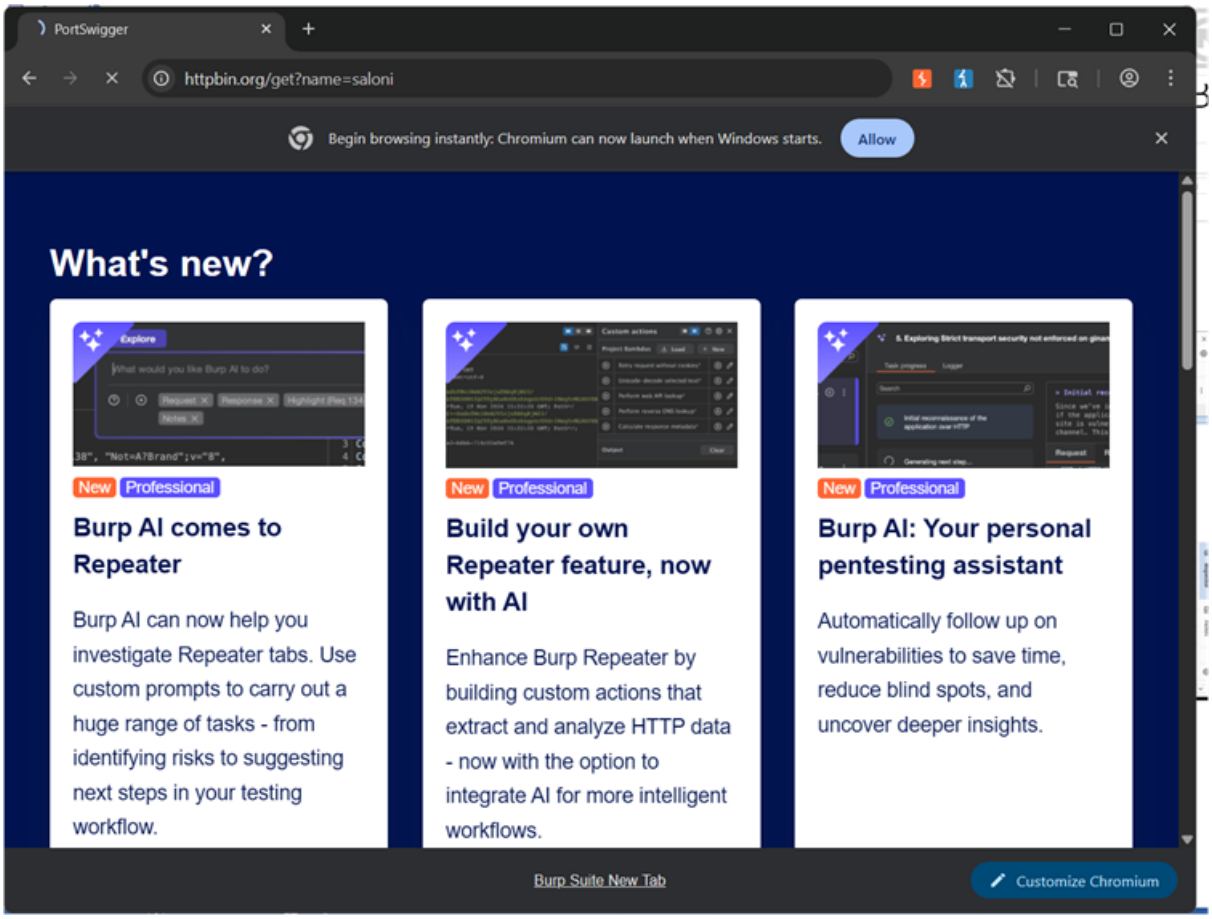
Screenshots:

```
┌──(kali㉿kali)-[~]
└─$ john
John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c 2021-11-02 10:45:52 +0100 OMP [linux-gnu 64-bit x86_64 AVX AC]
Copyright (c) 1996-2021 by Solar Designer and others
Homepage: https://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]

Use --help to list all available options.

┌──(kali㉿kali)-[~]
└─$ ls /usr/share/wordlists/
dirb  dirbuster  dnsmap.txt  fasttrack.txt  fern-wifi  john.lst  legion  metasploit  nmap.lst  rockyou.txt  sqlmap.txt  wfuzz  wifite.txt

┌──(kali㉿kali)-[~]
└─$ ls /usr/share/wordlists/rockyou.txt
/usr/share/wordlists/rockyou.txt

┌──(kali㉿kali)-[~]
└─$ echo "admin:$(openssl passwd -1 password123)" > ~/hash.txt

┌──(kali㉿kali)-[~]
└─$ cat ~/hash.txt
admin:$1$6OpnTX7I$svN4z./jVucIgMnx4Fd5B/

┌──(kali㉿kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt ~/hash.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 128/128 AVX 4×3])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password123      (admin)
1g 0:00:00:00 DONE (2026-05-10 08:21) 33.33g/s 51200p/s 51200c/s 51200C/s teacher..mexico1
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

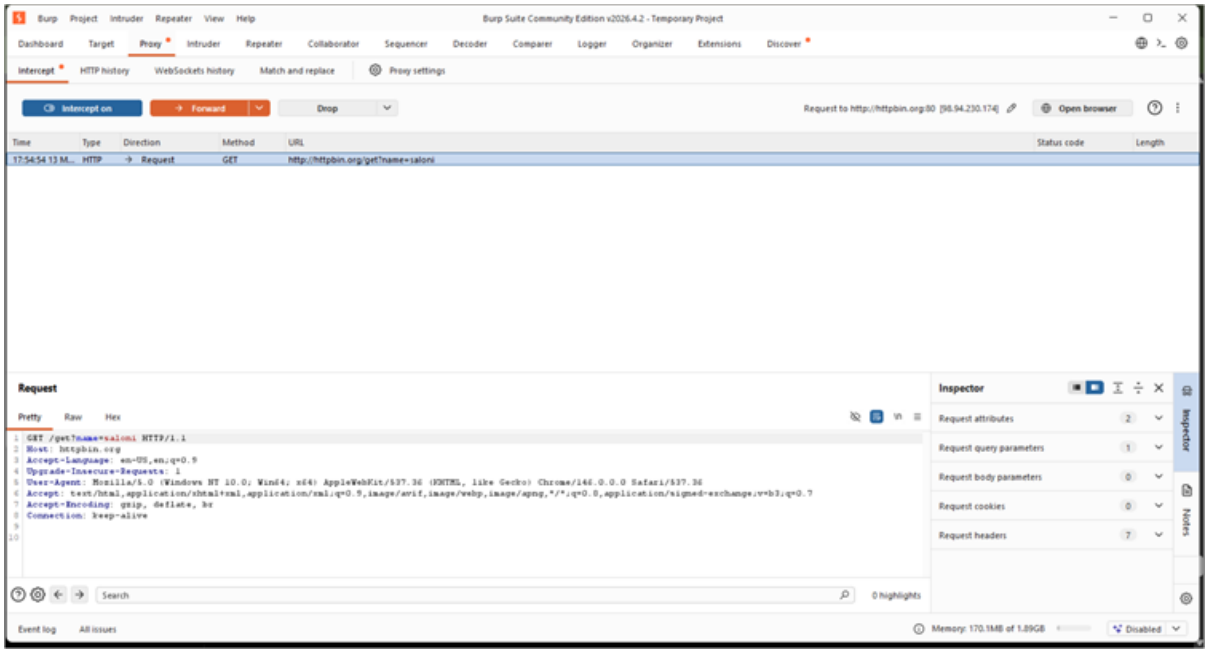
```
┌──(kali㉿kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt ~/hash.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 128/128 AVX 4×3])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password123      (admin)
1g 0:00:00:00 DONE (2026-05-10 08:21) 33.33g/s 51200p/s 51200c/s 51200C/s teacher..mexico1
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

┌──(kali㉿kali)-[~]
└─$ john --show ~/hash.txt
admin:password123

1 password hash cracked, 0 left
```

PRAC – 3:

Screenshots:

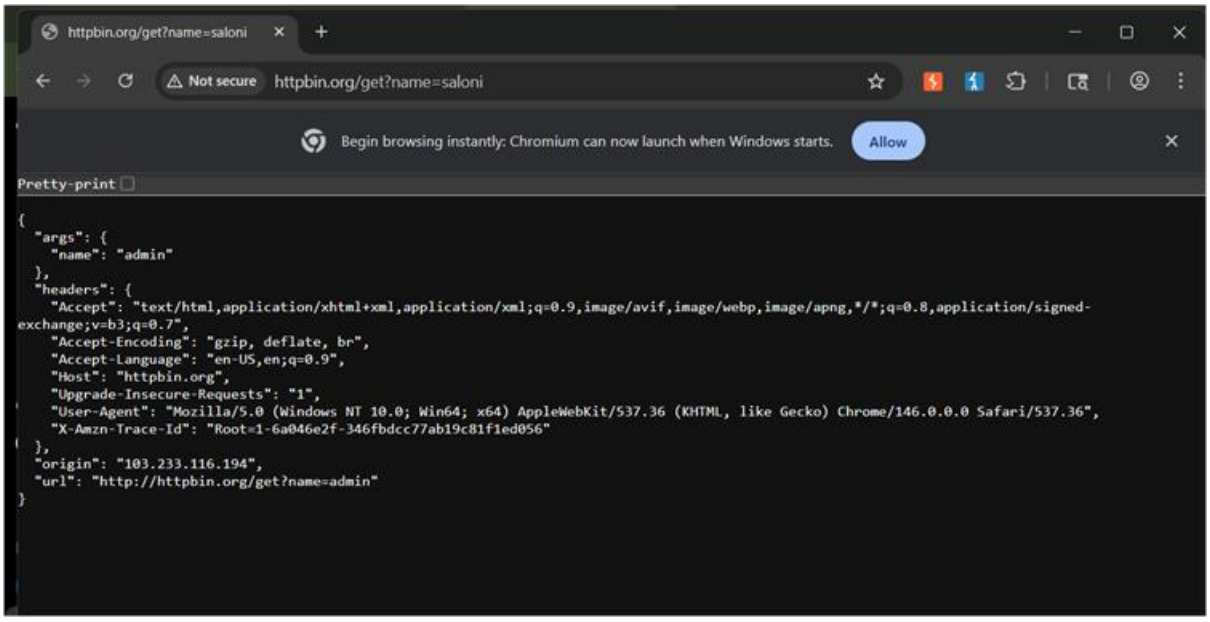


Request:

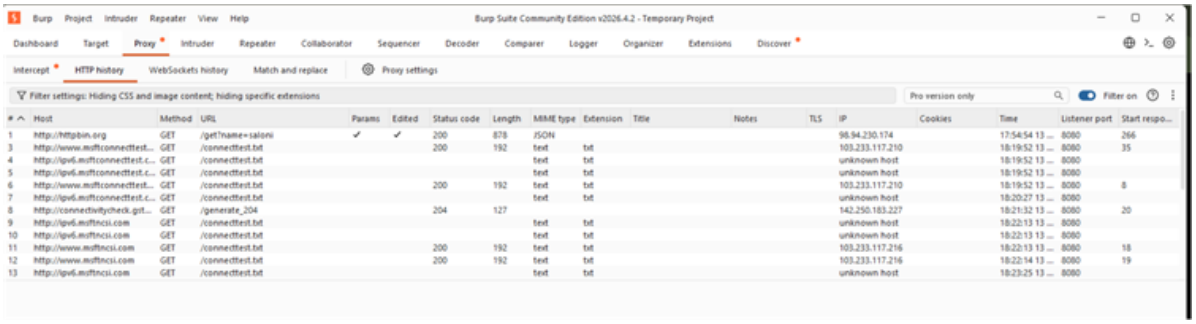


Modified request :

Changes made in the browser, name = admin now instead of saloni



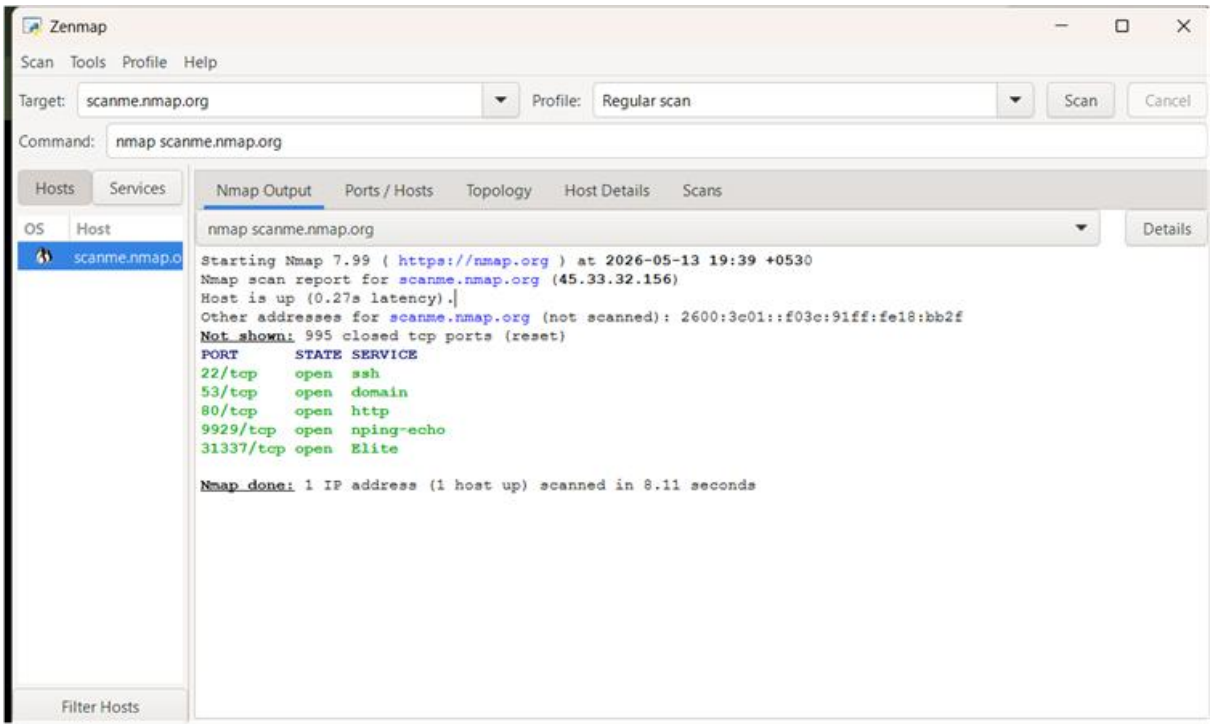
http history

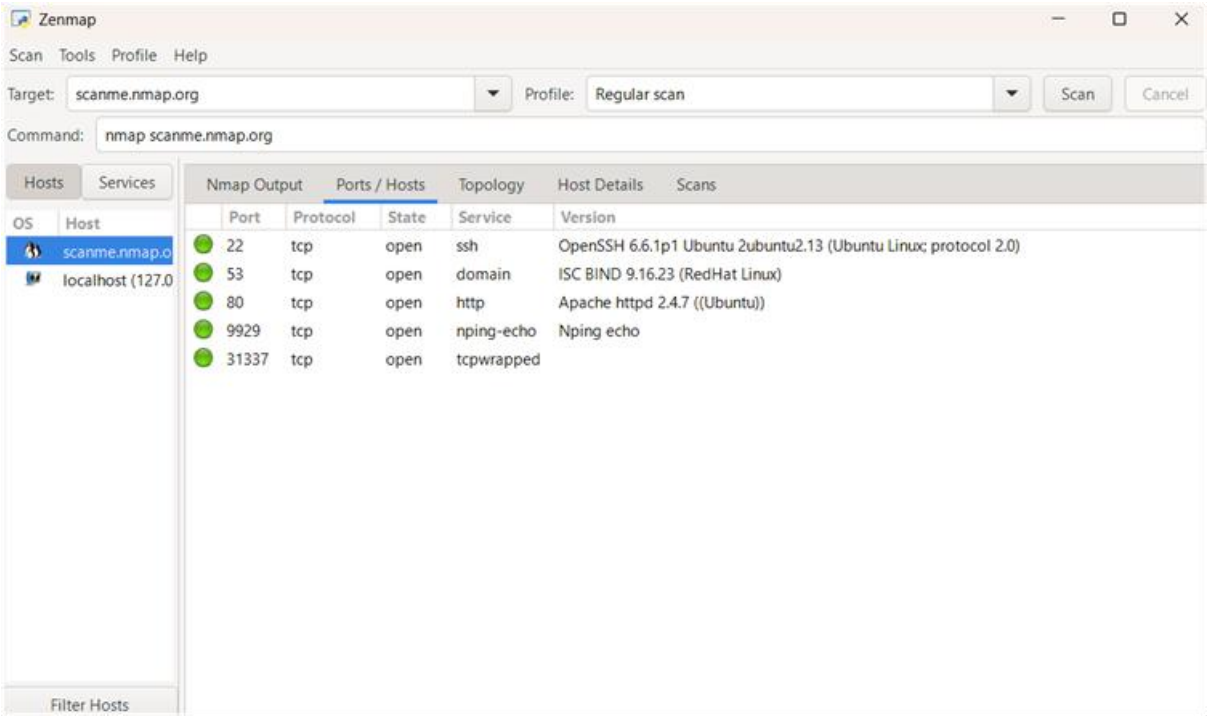


PRAC- 4:

Screenshots:

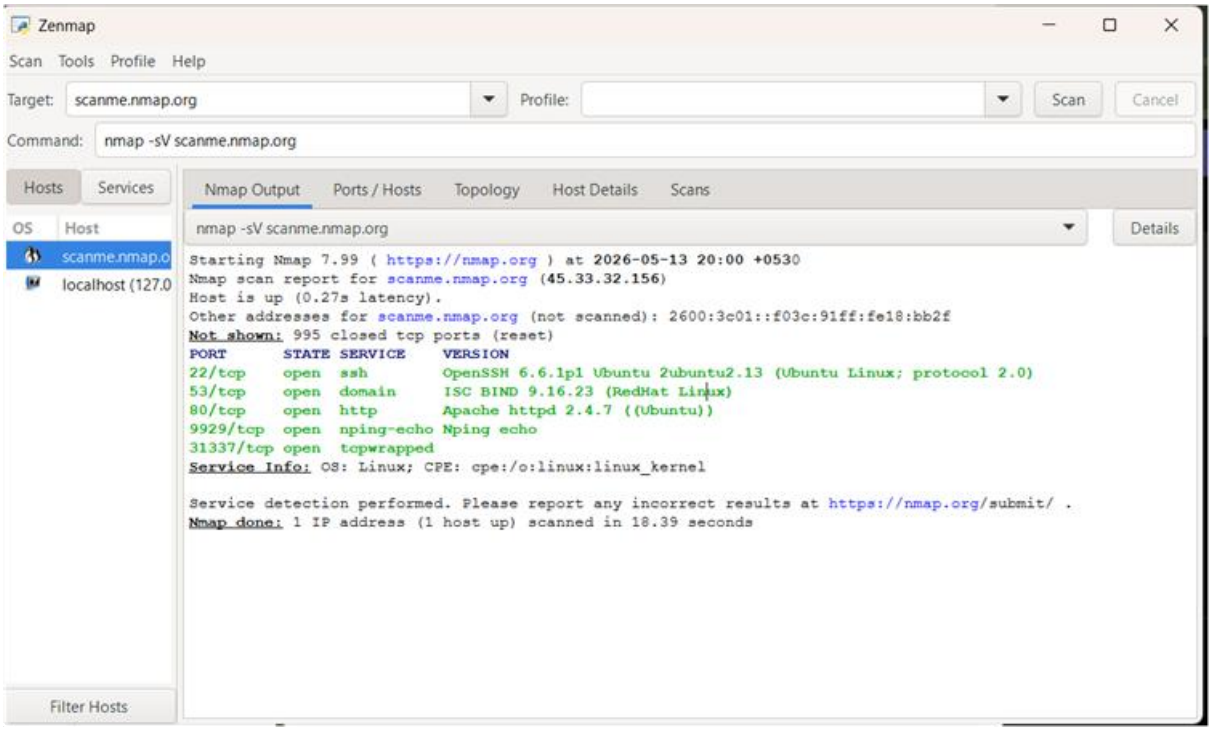
Regular scan:





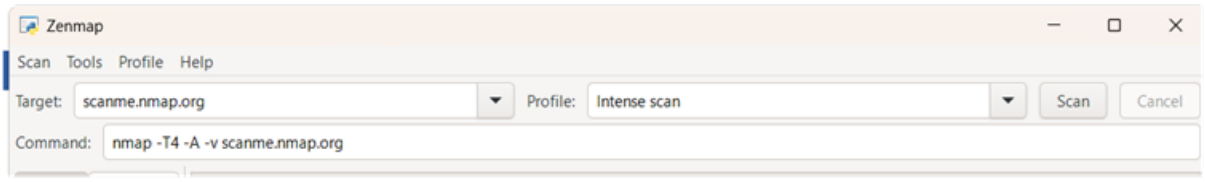
NMAP command:

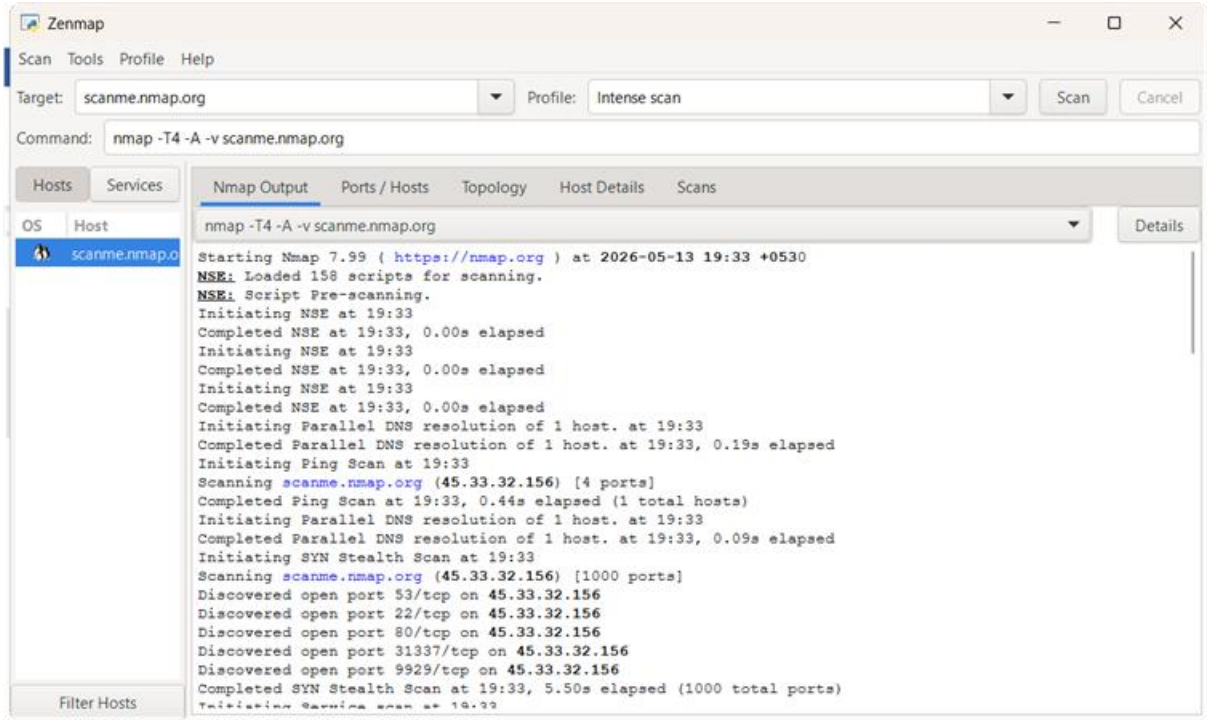
nmap -sV scanme.nmap.org

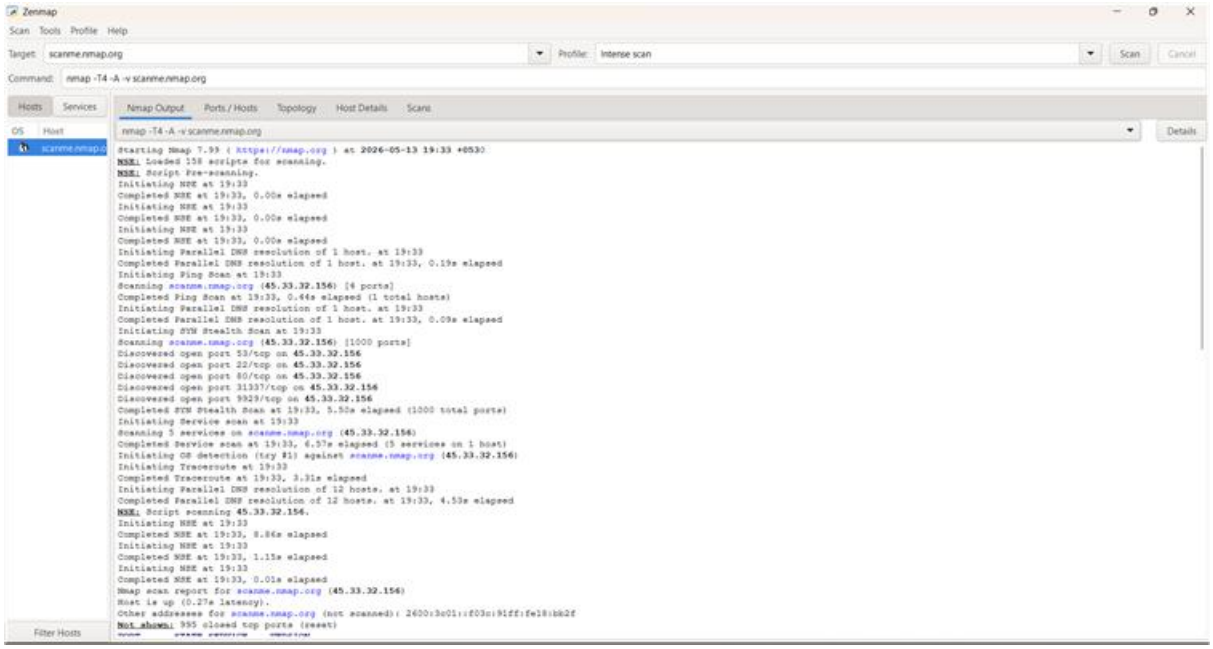


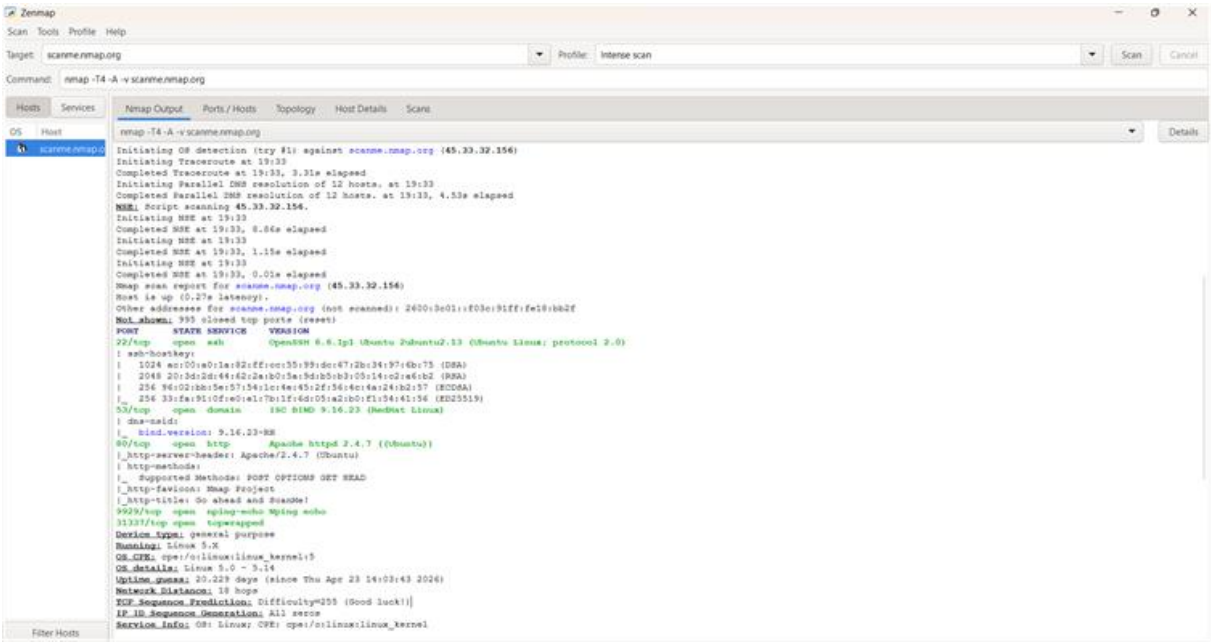
The -sV option in Nmap is used for service version detection. It identifies the services and their versions running on open ports.

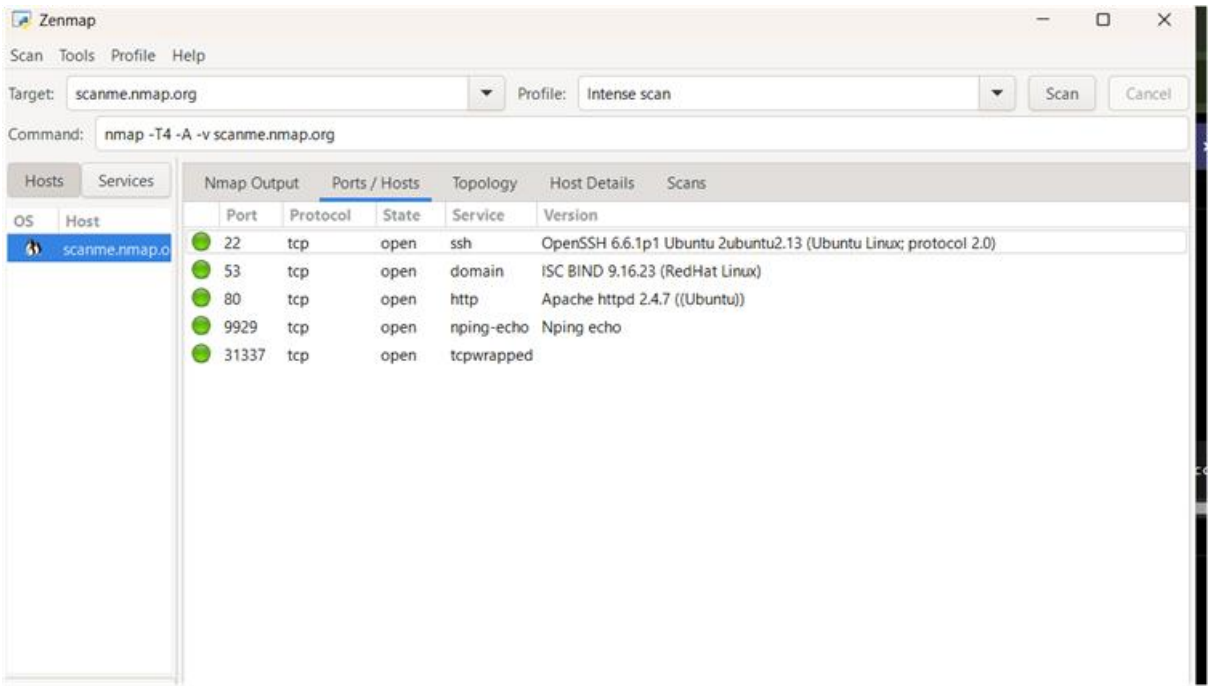
Intense scan:

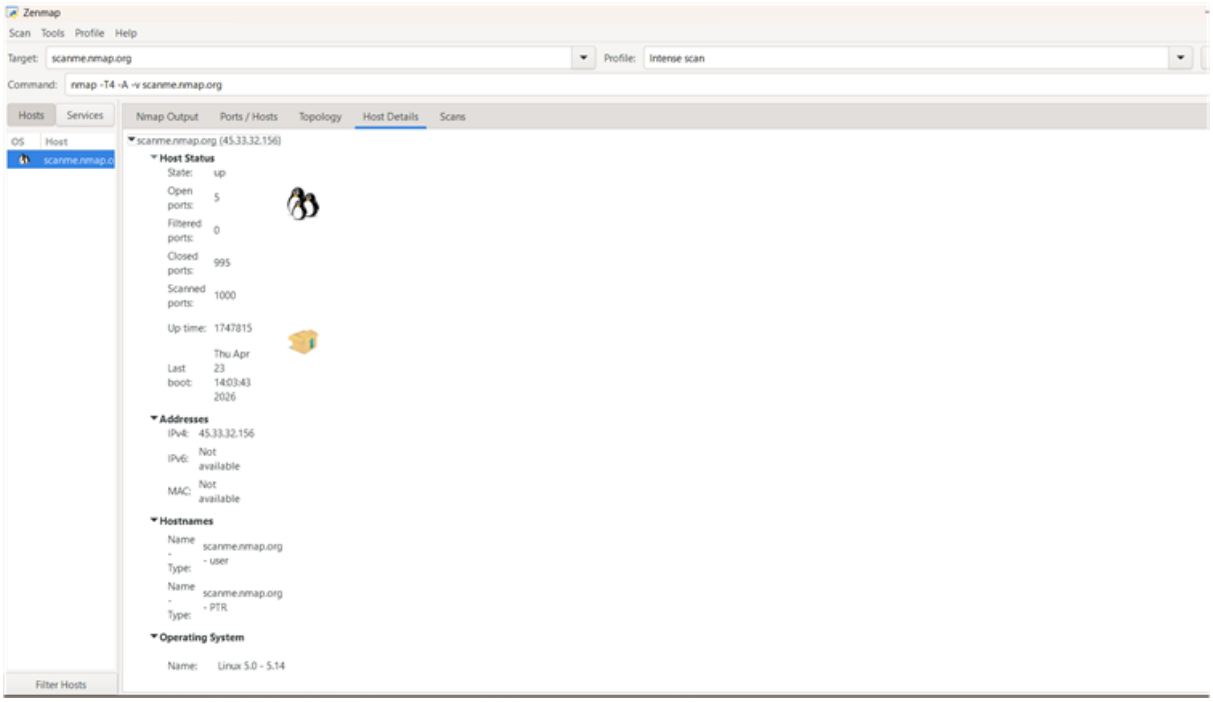



Zenmap
Scan Tools Profile Help
Target: scanme.nmap.org
Profile: Intense scan
Scan
Cancel
Command: nmap -T4 -A -v scanme.nmap.org
Hosts
Services
OS
Host
scanme.nmap.o
Filter Hosts
Nmap Output
Ports / Hosts
Topology
Host Details
Scans
nmap -T4 -A -v scanme.nmap.org
Details
Starting Nmap 7.99 (https://nmap.org) at 2026-05-13 19:33 +0530
NSE: Loaded 158 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 19:33
Completed NSE at 19:33, 0.00s elapsed
Initiating NSE at 19:33
Completed NSE at 19:33, 0.00s elapsed
Initiating NSE at 19:33
Completed NSE at 19:33, 0.00s elapsed
Initiating Parallel DNS resolution of 1 host. at 19:33
Completed Parallel DNS resolution of 1 host. at 19:33, 0.19s elapsed
Initiating Ping Scan at 19:33
Scanning scanme.nmap.org (45.33.32.156) [4 ports]
Completed Ping Scan at 19:33, 0.44s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 19:33
Completed Parallel DNS resolution of 1 host. at 19:33, 0.09s elapsed
Initiating SYN Stealth Scan at 19:33
Scanning scanme.nmap.org (45.33.32.156) [1000 ports]
Discovered open port 53/tcp on 45.33.32.156
Discovered open port 22/tcp on 45.33.32.156
Discovered open port 80/tcp on 45.33.32.156
Discovered open port 31337/tcp on 45.33.32.156
Discovered open port 9929/tcp on 45.33.32.156
Completed SYN Stealth Scan at 19:33, 5.50s elapsed (1000 total ports)




Zenmap
Scan Tools Profile Help
Target: scanme.nmap.org
Profile: Intense scan
Scan
Cancel
Command: nmap -T4 -A -v scanme.nmap.org
Hosts
Services
Nmap Output
Ports / Hosts
Topology
Host Details
Scans
OS Host
scanme.nmap.o
nmap -T4 -A -v scanme.nmap.org
Details
Initiating OS detection (try #1) against scanme.nmap.org (45.33.32.156)
Initiating Traceroute at 19:33
Completed Traceroute at 19:33, 3.31s elapsed
Initiating Parallel DNS resolution of 12 hosts. at 19:33
Completed Parallel DNS resolution of 12 hosts. at 19:33, 4.53s elapsed
NSE: Script scanning 45.33.32.156.
Initiating NSE at 19:33
Completed NSE at 19:33, 8.86s elapsed
Initiating NSE at 19:33
Completed NSE at 19:33, 1.15s elapsed
Initiating NSE at 19:33
Completed NSE at 19:33, 0.01s elapsed
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.27s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 995 closed tcp ports (reset)
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 1024 ac:00:a0:1a:82:ff:cc:55:99:dc:67:2b:34:97:6b:75 (DSA)
| 2048 20:3d:2d:44:62:2a:b0:5a:9d:b5:b3:05:14:c2:a6:b2 (RSA)
| 256 96:02:bb:5e:57:54:1c:4e:45:2f:56:4c:4a:24:b2:57 (ECDSA)
|_ 256 33:fa:91:0f:e0:e1:7b:1f:6d:05:a2:b0:f1:54:41:56 (ED25519)
53/tcp open domain ISC BIND 9.16.23 (RedHat Linux)
| dns-nsid:
|_ bind.version: 9.16.23-RH
80/tcp open http Apache httpd 2.4.7 ((Ubuntu))
|_http-server-header: Apache/2.4.7 (Ubuntu)
| http-methods:
|_ Supported Methods: POST OPTIONS GET HEAD
|_http-favicon: Nmap Project
|_http-title: Go ahead and ScanMe!
9929/tcp open nping-echo Nping echo
31337/tcp open tcpwrapped
Device type: general purpose
Running: Linux 5.X
OS CPE: cpe:/o:linux:linux_kernel:5
OS details: Linux 5.0 - 5.14
Uptime guess: 20.229 days (since Thu Apr 23 14:03:43 2026)
Network Distance: 19 hops
TCP Sequence Prediction: Difficulty=255 (Good luck!)
IP ID Sequence Generation: All zeros
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Filter Hosts


Zenmap
Scan Tools Profile Help
Target: scanme.nmap.org
Profile: Intense scan
Scan
Cancel
Command: nmap -T4 -A -v scanme.nmap.org
Hosts
Services
Nmap Output
Ports / Hosts
Topology
Host Details
Scans
OS Host
scanme.nmap.o
Port Protocol State Service Version
22 tcp open ssh OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
53 tcp open domain ISC BIND 9.16.23 (RedHat Linux)
80 tcp open http Apache httpd 2.4.7 ((Ubuntu))
9929 tcp open nping-echo Nping echo
31337 tcp open tcpwrapped



Observation: The Nmap scan successfully detected active ports and identified the services and versions running on the target machine.

PRAC - 5 :

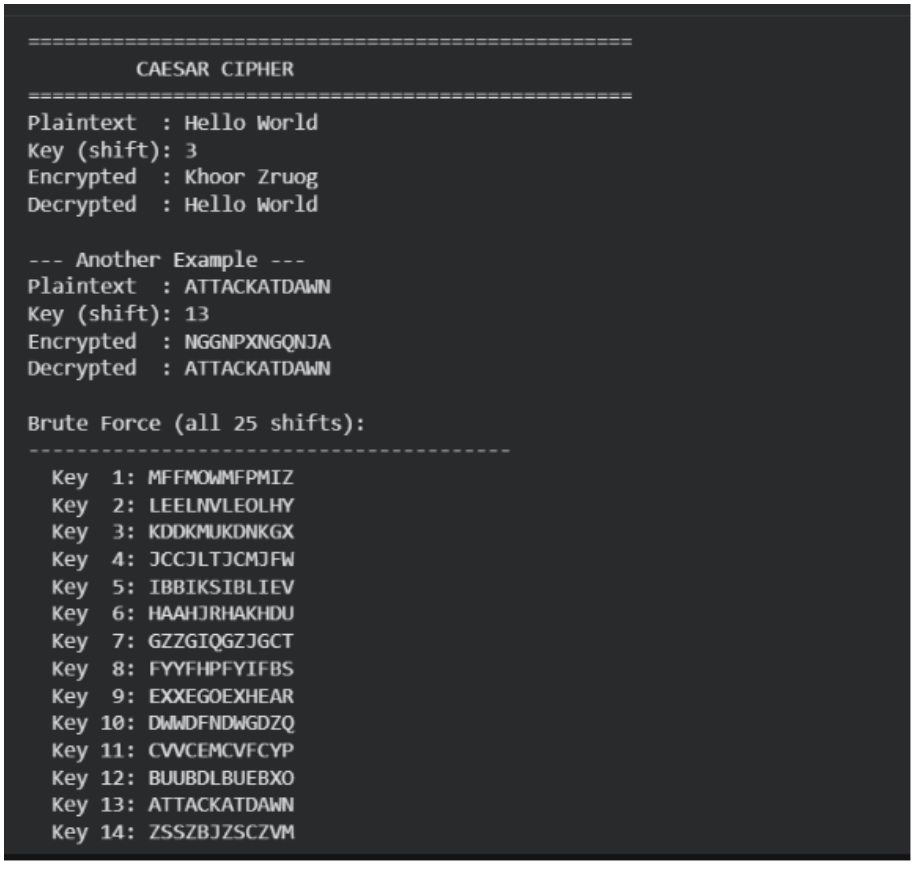
```
# ============================================================

# PRAC 5: Caesar Cipher Substitution Operation

# ============================================================

# Each letter is shifted by a fixed number (key) in the alphabet.

# Encryption: C = (P + key) mod 26

# Decryption: P = (C - key) mod 26

# ============================================================


def caesar_encrypt(plaintext, key):
```

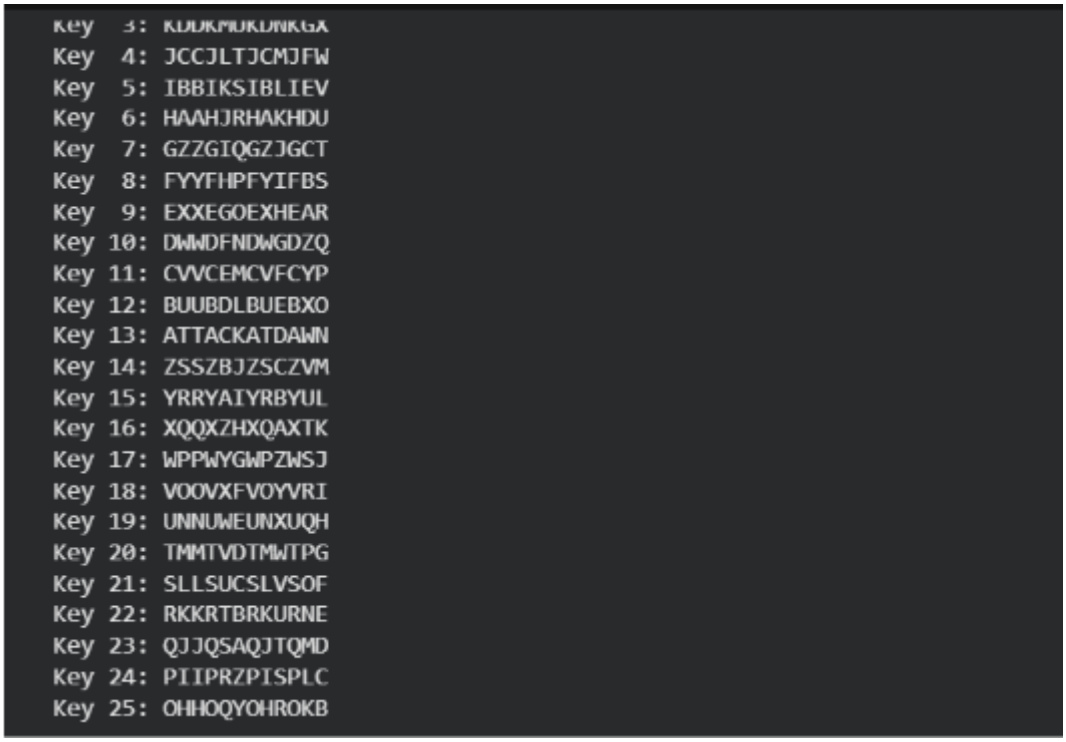
```
    result = ''
    for ch in plaintext:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + key) % 26 + base)
        else:
            result += ch   # spaces, digits, punctuation unchanged
    return result

def caesar_decrypt(ciphertext, key):
    return caesar_encrypt(ciphertext, -key)  # shift back by key

            def brute_force(ciphertext):
                """Try all 25 possible shifts."""
                print("\nBrute Force (all 25 shifts):")
                print("-" * 40)
                for shift in range(1, 26):
                    print(f"  Key {shift:2d}: {caesar_encrypt(ciphertext, -shift)}")

            # ==================== MAIN ====================
```

```
if __name__ == "__main__":
    print("=" * 50)
    print("        CAESAR CIPHER")
    print("=" * 50)

    plaintext = "Hello World"
    key = 3   # Classic Caesar uses key = 3

    encrypted = caesar_encrypt(plaintext, key)
    decrypted = caesar_decrypt(encrypted, key)

    print(f"Plaintext  : {plaintext}")
    print(f"Key (shift): {key}")
    print(f"Encrypted  : {encrypted}")
    print(f"Decrypted  : {decrypted}")

    print("\n--- Another Example ---")
    plaintext2 = "ATTACKATDAWN"
    key2 = 13  # ROT13 is Caesar with key=13
    encrypted2 = caesar_encrypt(plaintext2, key2)
    decrypted2 = caesar_decrypt(encrypted2, key2)
```

```
print(f"Plaintext  : {plaintext2}")

print(f"Key (shift): {key2}")

print(f"Encrypted  : {encrypted2}")

print(f"Decrypted  : {decrypted2}")


brute_force(encrypted2)
```

OUTPUT :





Prac - 6:

```
# ============================================================

# PRAC 6: Monoalphabetic and Polyalphabetic (Vigenere) Cipher

# ============================================================


# ---------- MONOALPHABETIC CIPHER ----------

# Uses a fixed substitution key (shuffled alphabet)


def mono_encrypt(plaintext, key):

    """
```

```
    key: a 26-letter string mapping a->key[0], b->key[1], ...
    """
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    result = ''
    for ch in plaintext.lower():
        if ch in alphabet:
            result += key[alphabet.index(ch)]
        else:
            result += ch
    return result

def mono_decrypt(ciphertext, key):
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    result = ''
    for ch in ciphertext.lower():
        if ch in key:
            result += alphabet[key.index(ch)]
        else:
            result += ch
```

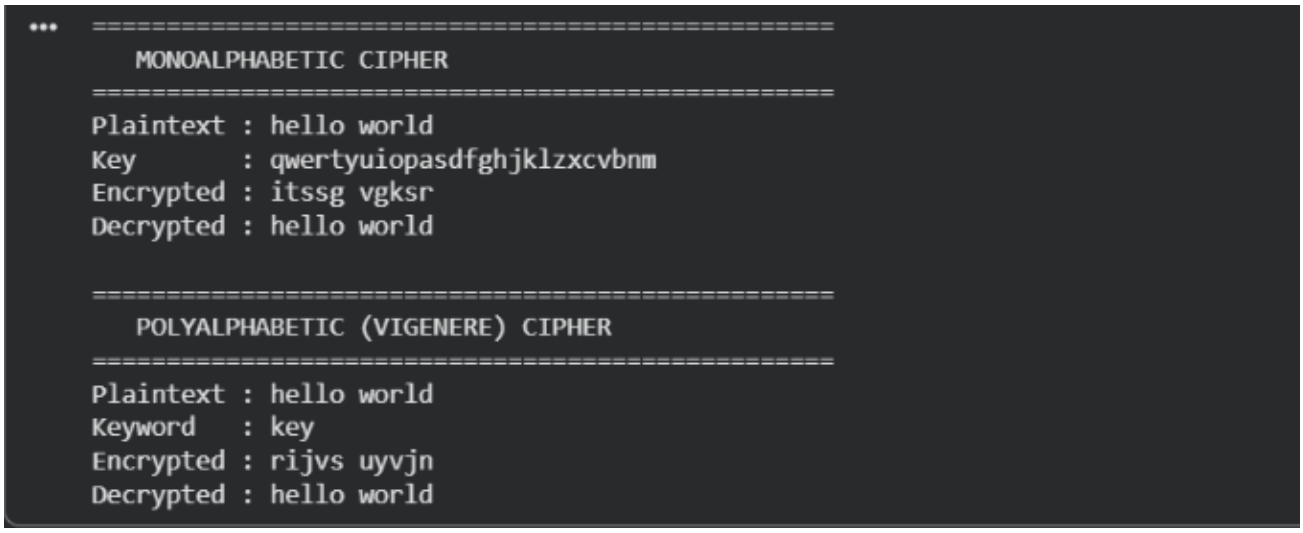
```
    return result


# ---------- POLYALPHABETIC (VIGENERE) CIPHER ----------

# Uses a keyword; each letter shifts by the keyword letter


def vigenere_encrypt(plaintext, keyword):

    alphabet = 'abcdefghijklmnopqrstuvwxyz'

    keyword = keyword.lower()

    result = ''

    k_idx = 0

    for ch in plaintext.lower():

        if ch in alphabet:

            shift = alphabet.index(keyword[k_idx % len(keyword)])

            result += alphabet[(alphabet.index(ch) + shift) % 26]

            k_idx += 1

        else:

            result += ch

    return result


def vigenere_decrypt(ciphertext, keyword):
```

```
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    keyword = keyword.lower()
    result = ''
    k_idx = 0
    for ch in ciphertext.lower():
        if ch in alphabet:
            shift = alphabet.index(keyword[k_idx % len(keyword)])
            result += alphabet[(alphabet.index(ch) - shift) % 26]
            k_idx += 1
        else:
            result += ch
    return result


# ==================== MAIN ====================
if __name__ == "__main__":
    print("=" * 50)
    print("   MONOALPHABETIC CIPHER")
    print("=" * 50)
    # Key: fixed substitution alphabet
```

```
mono_key = 'qwertyuiopasdfghjklzxcvbnm'

plaintext = "hello world"

encrypted = mono_encrypt(plaintext, mono_key)

decrypted = mono_decrypt(encrypted, mono_key)

print(f"Plaintext : {plaintext}")

print(f"Key       : {mono_key}")

print(f"Encrypted : {encrypted}")

print(f"Decrypted : {decrypted}")


print("\n" + "=" * 50)

print("    POLYALPHABETIC (VIGENERE) CIPHER")

print("=" * 50)

plaintext = "hello world"

keyword   = "key"

encrypted = vigenere_encrypt(plaintext, keyword)

decrypted = vigenere_decrypt(encrypted, keyword)

print(f"Plaintext : {plaintext}")

print(f"Keyword   : {keyword}")

print(f"Encrypted : {encrypted}")
```

```
    print(f"Decrypted : {decrypted}")
```

OUTPUT :



PRAC-7 :

```
# ============================================================

# PRAC 7: Playfair Cipher Substitution

# ============================================================


def generate_matrix(key):

    key = key.upper().replace('J', 'I')

    seen = []

    for ch in key:
```

```
        if ch not in seen and ch.isalpha():
            seen.append(ch)
    for ch in 'ABCDEFGHIKLMNOPQRSTUVWXYZ':
        if ch not in seen:
            seen.append(ch)
    matrix = [seen[i*5:(i+1)*5] for i in range(5)]
    return matrix

def find_position(matrix, ch):
    for r in range(5):
        for c in range(5):
            if matrix[r][c] == ch:
                return r, c
    return None

def prepare_text(plaintext):
    text = plaintext.upper().replace('J', 'I')
    text = ''.join(filter(str.isalpha, text))
    result = ''
    i = 0
```

```
    while i < len(text):
        a = text[i]
        if i + 1 < len(text):
            b = text[i + 1]
        else:
            b = 'X'
        if a == b:
            result += a + 'X'
            i += 1
        else:
            result += a + b
            i += 2
    if len(result) % 2 != 0:
        result += 'X'
    return result

def playfair_encrypt(plaintext, key):
    matrix = generate_matrix(key)
    text = prepare_text(plaintext)
```

```
    ciphertext = ''
    for i in range(0, len(text), 2):
        r1, c1 = find_position(matrix, text[i])
        r2, c2 = find_position(matrix, text[i+1])
        if r1 == r2:                    # same row
            ciphertext += matrix[r1][(c1+1)%5] + matrix[r2][(c2+1)%5]
        elif c1 == c2:                  # same column
            ciphertext += matrix[(r1+1)%5][c1] + matrix[(r2+1)%5][c2]
        else:                           # rectangle
            ciphertext += matrix[r1][c2] + matrix[r2][c1]
    return ciphertext

def playfair_decrypt(ciphertext, key):
    matrix = generate_matrix(key)
    ciphertext = ciphertext.upper()
    plaintext = ''
    for i in range(0, len(ciphertext), 2):
        r1, c1 = find_position(matrix, ciphertext[i])
        r2, c2 = find_position(matrix, ciphertext[i+1])
```

```
        if r1 == r2:
            plaintext += matrix[r1][(c1-1)%5] + matrix[r2][(c2-1)%5]
        elif c1 == c2:
            plaintext += matrix[(r1-1)%5][c1] + matrix[(r2-1)%5][c2]
        else:
            plaintext += matrix[r1][c2] + matrix[r2][c1]
    return plaintext

def print_matrix(matrix):
    print("Playfair Matrix:")
    for row in matrix:
        print(' '.join(row))

# ==================== MAIN ====================

if __name__ == "__main__":
    print("=" * 50)
    print("   PLAYFAIR CIPHER")
    print("=" * 50)
    key = "MONARCHY"
    plaintext = "INSTRUMENTS"
```

```
matrix = generate_matrix(key)

print_matrix(matrix)

print()


encrypted = playfair_encrypt(plaintext, key)

decrypted = playfair_decrypt(encrypted, key)


print(f"Plaintext  : {plaintext}")

print(f"Key        : {key}")

print(f"Prepared   : {prepare_text(plaintext)}")

print(f"Encrypted  : {encrypted}")

print(f"Decrypted  : {decrypted}")
```

OUTPUT :

```
==================================================
   PLAYFAIR CIPHER
==================================================
Playfair Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Plaintext  : INSTRUMENTS
Key        : MONARCHY
Prepared   : INSTRUMENTSX
Encrypted  : GATLMZCLRQXA
Decrypted  : INSTRUMENTSX
```

PRAC-8:

```
# ============================================================

# PRAC 8: Hill Cipher Substitution

# ============================================================

import numpy as np


def mod_inverse(a, m=26):

    """Extended Euclidean algorithm to find modular inverse."""

    for x in range(1, m):

        if (a * x) % m == 1:

            return x

    return None


def matrix_mod_inverse(matrix, mod=26):

    """Find modular inverse of a matrix (mod 26)."""

    det = int(round(np.linalg.det(matrix))) % mod

    det_inv = mod_inverse(det % mod, mod)

    if det_inv is None:

        raise ValueError("Matrix is not invertible mod 26")
```

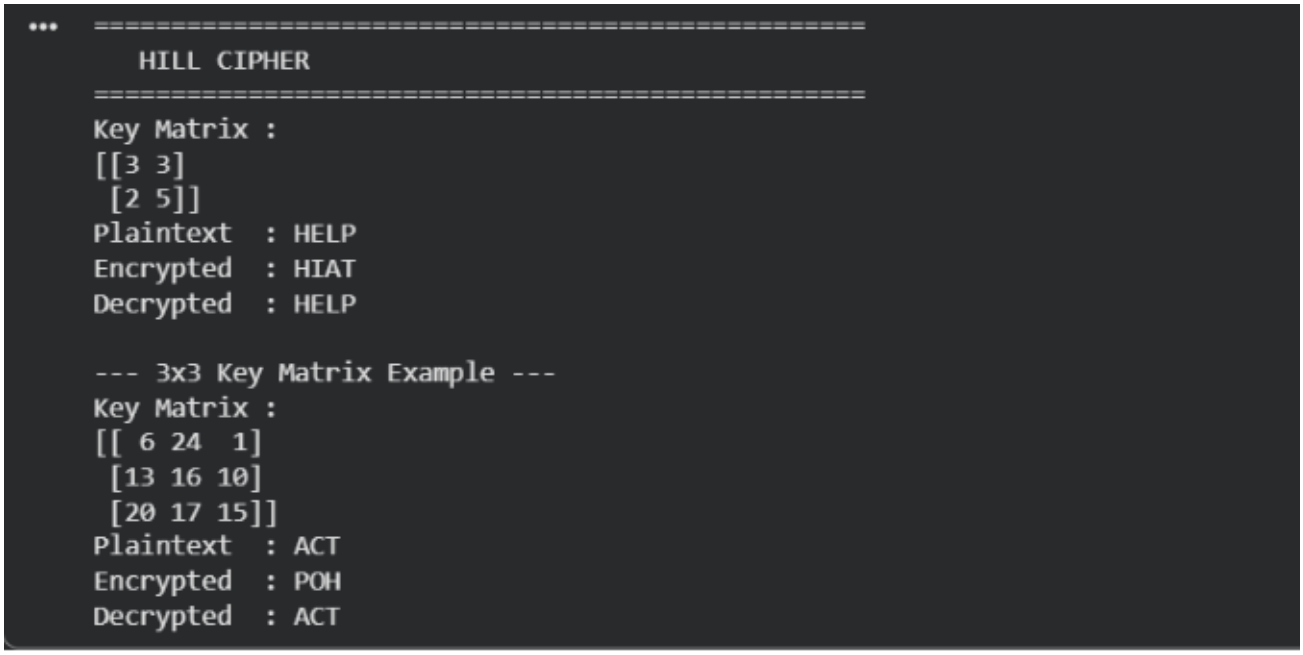
```
    size = matrix.shape[0]
    # Adjugate (cofactor transpose)
    cofactors = np.zeros_like(matrix)
    for r in range(size):
        for c in range(size):
            minor = np.delete(np.delete(matrix, r, axis=0), c, axis=1)
            cofactors[r][c] = ((-1)**(r+c)) * round(np.linalg.det(minor))
    adjugate = cofactors.T
    inv_matrix = (det_inv * adjugate) % mod
    return inv_matrix.astype(int)

def text_to_vector(text):
    return [ord(ch) - ord('A') for ch in text.upper() if ch.isalpha()]

def vector_to_text(vector):
    return ''.join(chr(int(v) % 26 + ord('A')) for v in vector)

def hill_encrypt(plaintext, key_matrix):
    n = key_matrix.shape[0]
    text = ''.join(filter(str.isalpha, plaintext.upper()))
```

```
    # Pad to multiple of n
    while len(text) % n != 0:
        text += 'X'
    ciphertext = ''
    for i in range(0, len(text), n):
        block = np.array(text_to_vector(text[i:i+n]))
        encrypted_block = np.dot(key_matrix, block) % 26
        ciphertext += vector_to_text(encrypted_block)
    return ciphertext

def hill_decrypt(ciphertext, key_matrix):
    n = key_matrix.shape[0]
    inv_key = matrix_mod_inverse(key_matrix)
    plaintext = ''
    for i in range(0, len(ciphertext), n):
        block = np.array(text_to_vector(ciphertext[i:i+n]))
        decrypted_block = np.dot(inv_key, block) % 26
        plaintext += vector_to_text(decrypted_block)
    return plaintext
```

```
# ==================== MAIN ====================

if __name__ == "__main__":

    print("=" * 50)

    print("   HILL CIPHER")

    print("=" * 50)


    # 2x2 key matrix

    key_matrix = np.array([[3, 3],

                           [2, 5]])


    plaintext = "HELP"


    print(f"Key Matrix :\n{key_matrix}")

    print(f"Plaintext  : {plaintext}")


    encrypted = hill_encrypt(plaintext, key_matrix)

    decrypted = hill_decrypt(encrypted, key_matrix)


    print(f"Encrypted  : {encrypted}")

    print(f"Decrypted  : {decrypted}")


    print("\n--- 3x3 Key Matrix Example ---")
```

```
key_matrix_3 = np.array([[6,  24, 1],

                [13, 16, 10],

                [20, 17, 15]])

plaintext2 = "ACT"

encrypted2 = hill_encrypt(plaintext2, key_matrix_3)

decrypted2 = hill_decrypt(encrypted2, key_matrix_3)

print(f"Key Matrix :\n{key_matrix_3}")

print(f"Plaintext  : {plaintext2}")

print(f"Encrypted  : {encrypted2}")

print(f"Decrypted  : {decrypted2}")
```

OUTPUT :



PRAC -9 :

```
# ============================================================

# PRAC 9: Rail Fence Cipher (Transposition)

# ============================================================


def rail_fence_encrypt(plaintext, rails):

    """

    Write text in zigzag across 'rails' rows, then read row by row.

    """

    fence = [[] for _ in range(rails)]

    rail = 0

    direction = 1  # 1 = down, -1 = up


    for ch in plaintext:

        fence[rail].append(ch)

        if rail == 0:

            direction = 1

        elif rail == rails - 1:

            direction = -1
```

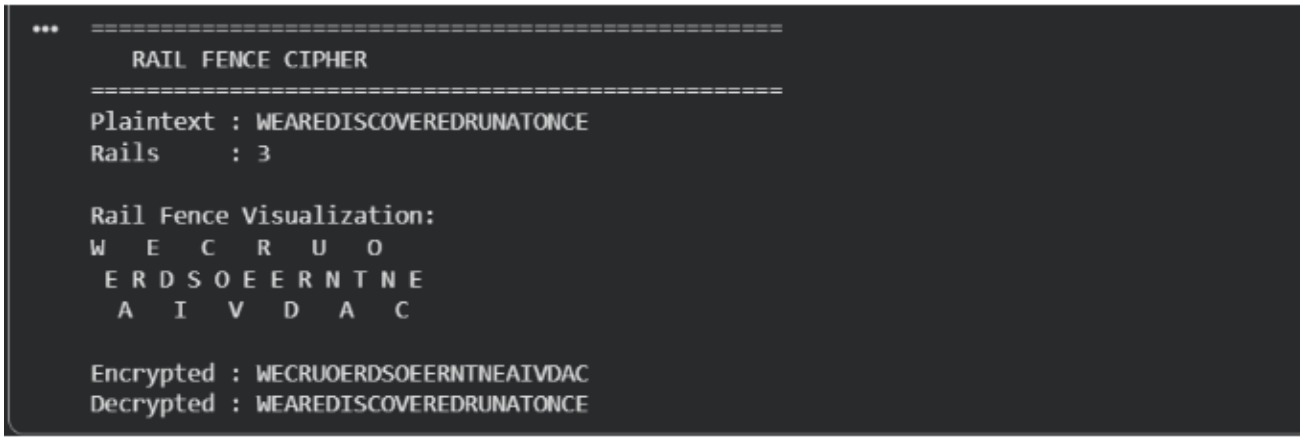
```
        rail += direction

    ciphertext = ''.join(''.join(row) for row in fence)
    return ciphertext

def rail_fence_decrypt(ciphertext, rails):
    """
    Reconstruct the fence pattern, mark positions, then fill and read.
    """
    n = len(ciphertext)
    # Step 1: Mark which rail each position belongs to
    pattern = []
    rail = 0
    direction = 1
    for i in range(n):
        pattern.append(rail)
        if rail == 0:
            direction = 1
        elif rail == rails - 1:
            direction = -1
```

```
        rail += direction

    # Step 2: Count chars per rail
    counts = [pattern.count(r) for r in range(rails)]

    # Step 3: Split ciphertext into rails
    fence = []
    idx = 0
    for count in counts:
        fence.append(list(ciphertext[idx:idx+count]))
        idx += count

    # Step 4: Read in zigzag order
    rail_idx = [0] * rails
    plaintext = ''
    for r in pattern:
        plaintext += fence[r][rail_idx[r]]
        rail_idx[r] += 1

    return plaintext

def show_fence(plaintext, rails):
```

```
    """Display the zigzag fence visually."""
    fence = [[' '] * len(plaintext) for _ in range(rails)]
    rail = 0
    direction = 1
    for i, ch in enumerate(plaintext):
        fence[rail][i] = ch
        if rail == 0:
            direction = 1
        elif rail == rails - 1:
            direction = -1
        rail += direction
    print("Rail Fence Visualization:")
    for row in fence:
        print(''.join(row))

# ==================== MAIN ====================
if __name__ == "__main__":
    print("=" * 50)
    print("   RAIL FENCE CIPHER")
```

```
print("=" * 50)

plaintext = "WEAREDISCOVEREDRUNATONCE"

rails = 3


print(f"Plaintext : {plaintext}")

print(f"Rails     : {rails}\n")

show_fence(plaintext, rails)


encrypted = rail_fence_encrypt(plaintext, rails)

decrypted = rail_fence_decrypt(encrypted, rails)


print(f"\nEncrypted : {encrypted}")

print(f"Decrypted : {decrypted}")
```

OUTPUT :



PRAC - 10:

```
# ============================================================

# PRAC 10: Row Transposition Cipher (Columnar Transposition)

# ============================================================

import math


def row_transpose_encrypt(plaintext, key):

    """

    Write plaintext in rows of len(key) columns.

    Read columns in the order defined by the sorted key.

    """

    key = key.upper()

    n_cols = len(key)

    # Pad with 'X' if needed

    padded = plaintext.upper().replace(' ', '')

    while len(padded) % n_cols != 0:

        padded += 'X'

    n_rows = len(padded) // n_cols


    # Build the grid

    grid = [list(padded[i*n_cols:(i+1)*n_cols]) for i in range(n_rows)]
```

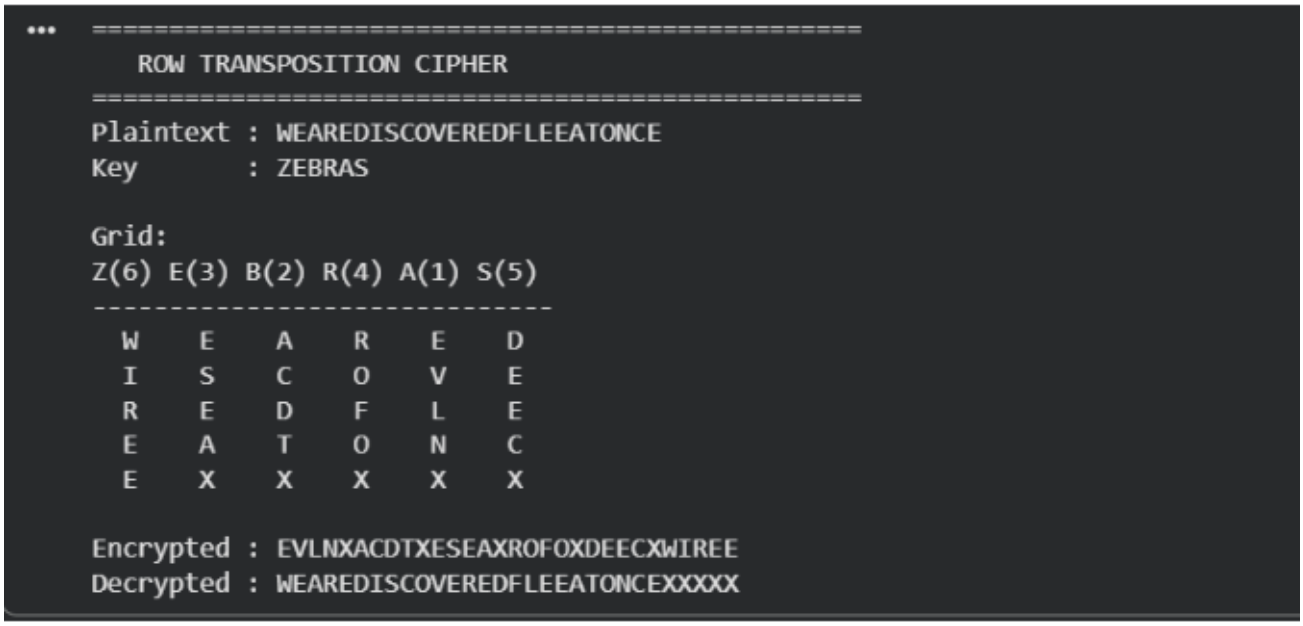
```
    # Determine column read order by sorting key alphabetically
    order = sorted(range(n_cols), key=lambda x: key[x])

    ciphertext = ''
    for col in order:
        for row in grid:
            ciphertext += row[col]
    return ciphertext

def row_transpose_decrypt(ciphertext, key):
    """
    Reverse the columnar transposition.
    """
    key = key.upper()
    n_cols = len(key)
    n_rows = len(ciphertext) // n_cols
    order = sorted(range(n_cols), key=lambda x: key[x])

    # Rebuild columns
    cols = {}
```

```
    idx = 0
    for col in order:
        cols[col] = list(ciphertext[idx:idx+n_rows])
        idx += n_rows

    # Read row by row
    plaintext = ''
    for r in range(n_rows):
        for c in range(n_cols):
            plaintext += cols[c][r]
    return plaintext

def print_grid(text, key):
    n_cols = len(key)
    padded = text.upper().replace(' ', '')
    while len(padded) % n_cols != 0:
        padded += 'X'
    n_rows = len(padded) // n_cols
    order = sorted(range(n_cols), key=lambda x: key.upper()[x])
    key_order = [order.index(i)+1 for i in range(n_cols)]
```

```
    print("Grid:")
    print(' '.join(f"{k}({o})" for k, o in zip(key.upper(), key_order)))
    print('-' * (n_cols * 5))
    for i in range(n_rows):
        print(' '.join(f"  {padded[i*n_cols+j]} " for j in range(n_cols)))

# ==================== MAIN ====================
if __name__ == "__main__":
    print("=" * 50)
    print("   ROW TRANSPOSITION CIPHER")
    print("=" * 50)
    plaintext = "WEAREDISCOVEREDFLEEATONCE"
    key = "ZEBRAS"

    print(f"Plaintext : {plaintext}")
    print(f"Key       : {key}\n")
    print_grid(plaintext, key)

    encrypted = row_transpose_encrypt(plaintext, key)
    decrypted = row_transpose_decrypt(encrypted, key)
```

```
print(f"\nEncrypted : {encrypted}")

print(f"Decrypted : {decrypted}")
```

OUTPUT :



PRAC - 11:

```
# ============================================================

# PRAC 11: Product Cipher (Transposition × Transposition)

# Combines Rail Fence + Row Transposition in sequence

# ============================================================


# -------- Rail Fence (from Prac 9) --------

def rail_fence_encrypt(plaintext, rails):
```

```
    fence = [[] for _ in range(rails)]
    rail, direction = 0, 1
    for ch in plaintext:
        fence[rail].append(ch)
        if rail == 0:          direction = 1
        elif rail == rails - 1: direction = -1
        rail += direction
    return ''.join(''.join(row) for row in fence)

def rail_fence_decrypt(ciphertext, rails):
    n = len(ciphertext)
    pattern = []
    rail, direction = 0, 1
    for _ in range(n):
        pattern.append(rail)
        if rail == 0:          direction = 1
        elif rail == rails - 1: direction = -1
        rail += direction
    counts = [pattern.count(r) for r in range(rails)]
```

```
    fence, idx = [], 0
    for count in counts:
        fence.append(list(ciphertext[idx:idx+count]))
        idx += count
    rail_idx = [0] * rails
    plaintext = ''
    for r in pattern:
        plaintext += fence[r][rail_idx[r]]
        rail_idx[r] += 1
    return plaintext

# -------- Row Transposition (from Prac 10) --------
def row_transpose_encrypt(plaintext, key):
    key = key.upper()
    n_cols = len(key)
    padded = plaintext.upper().replace(' ', '')
    while len(padded) % n_cols != 0:
        padded += 'X'
    n_rows = len(padded) // n_cols
```

```
    grid = [list(padded[i*n_cols:(i+1)*n_cols]) for i in range(n_rows)]
    order = sorted(range(n_cols), key=lambda x: key[x])
    ciphertext = ''
    for col in order:
        for row in grid:
            ciphertext += row[col]
    return ciphertext

def row_transpose_decrypt(ciphertext, key):
    key = key.upper()
    n_cols = len(key)
    n_rows = len(ciphertext) // n_cols
    order = sorted(range(n_cols), key=lambda x: key[x])
    cols, idx = {}, 0
    for col in order:
        cols[col] = list(ciphertext[idx:idx+n_rows])
        idx += n_rows
    plaintext = ''
    for r in range(n_rows):
```

```
        for c in range(n_cols):
            plaintext += cols[c][r]
    return plaintext

# -------- Product Cipher --------
def product_encrypt(plaintext, rails, key):
    """
    Stage 1: Rail Fence Encryption
    Stage 2: Row Transposition Encryption
    """
    stage1 = rail_fence_encrypt(plaintext.upper().replace(' ', ''), rails)
    stage2 = row_transpose_encrypt(stage1, key)
    return stage1, stage2

def product_decrypt(ciphertext, rails, key):
    """
    Stage 1: Row Transposition Decryption
    Stage 2: Rail Fence Decryption (strip trailing X padding first)
    """
    stage1 = row_transpose_decrypt(ciphertext, key)
```

```
    # Strip padding Xs added during row transposition
    stage1_stripped = stage1.rstrip('X')
    # Restore to original encrypted length (before row-transpose padding)
    stage2 = rail_fence_decrypt(stage1_stripped, rails)
    return stage1_stripped, stage2

# ==================== MAIN ====================
if __name__ == "__main__":
    print("=" * 55)
    print("   PRODUCT CIPHER  (Rail Fence + Row Transposition)")
    print("=" * 55)

    plaintext = "WEAREDISCOVEREDFLEEATONCE"
    rails = 3
    key   = "HACK"

    print(f"Plaintext       : {plaintext}")
    print(f"Rails           : {rails}")
    print(f"Transposition Key: {key}")

    print("\n--- ENCRYPTION ---")
```

```
after_rail, final_cipher = product_encrypt(plaintext, rails, key)

print(f"After Rail Fence : {after_rail}")

print(f"Final Ciphertext : {final_cipher}")


print("\n--- DECRYPTION ---")

after_row, recovered = product_decrypt(final_cipher, rails, key)

print(f"After Row Transp : {after_row}")

print(f"Recovered Text   : {recovered}")


print("\n--- VERIFICATION ---")

original = plaintext.upper().replace(' ', '')

match = original == recovered[:len(original)]

print(f"Original  : {original}")

print(f"Recovered : {recovered}")

print(f"Match     : {'✓ YES' if match else '✗ NO'}")
```

OUTPUT :

```
...  ======================================================
        PRODUCT CIPHER  (Rail Fence + Row Transposition)
     ======================================================
     Plaintext        : WEAREDISCOVEREDFLEEATONCE
     Rails            : 3
     Transposition Key: HACK

     --- ENCRYPTION ---
     After Rail Fence : WECRLTEERDSOEEFEAOCAIVDEN
     Final Ciphertext : ETDEOVXCESFCDXWLREAINREOEAEX

     --- DECRYPTION ---
     After Row Transp : WECRLTEERDSOEEFEAOCAIVDEN
     Recovered Text   : WEAREDISCOVEREDFLEEATONCE

     --- VERIFICATION ---
     Original  : WEAREDISCOVEREDFLEEATONCE
     Recovered : WEAREDISCOVEREDFLEEATONCE
     Match     : ✓ YES
```